

Московский государственный университет имени М.В. Ломоносова
Физический факультет
Кафедра общей физики и волновых процессов

Отчет по практическому заданию №2
Основы математического моделирования

Выполнил студент 325 группы
Делекторский Никита Юрьевич

Москва
2023

Оглавление

I. Постановка задачи (задача № 70)	3
II. Аналитическое решение	3
III. Построение разностной схемы.....	4
IV. Метод прогонки.....	6
V. Реализация метода переменных направлений	7
VI. Результаты	7
VII. Сравнение результатов с аналитическим решением.....	9
VIII. Заключение	12
Приложение 1	13

I. Постановка задачи (задача № 70)

Используя метод переменных направлений, решите краевую задачу:

$$\begin{cases} \partial_t u = \Delta u + e^t \sin x \cos y & \begin{cases} 0 < x < \pi \\ 0 < y < \pi \\ t > 0 \end{cases} \\ u|_{x=0} = u|_{x=\pi} = 0 \\ \partial_y u|_{y=0} = \partial_y u|_{y=\pi} = 0 \\ u|_{t=0} = \sin 2x \cos y \end{cases} \quad (1)$$

II. Аналитическое решение

Для начала требуется найти аналитическое решение данной задачи. Из постановки задачи видно, что начальное условие и неоднородность ортогональны. Поэтому решение будем искать в виде:

$$u(x, y, t) = \sum_m T_{nm}(t) \sum_n V_{nm}(x, y) \quad (2)$$

Поставим задачу Штурма-Лиувилля в прямоугольной области и разделим переменные, получив тем самым явный вид функций $T_{nm}(t)$ и $V_{nm}(x, y)$:

$$\left\{ \begin{array}{l} \Delta V + \lambda V = 0 \\ V|_{x=0} = V|_{x=\pi} = 0 \\ \partial_y V|_{y=0} = \partial_y V|_{y=\pi} = 0 \end{array} \right. \quad \begin{array}{l} 0 < x < \pi \\ 0 < y < \pi \end{array} \quad \left\{ \begin{array}{l} \partial_t T + \lambda T = f(t) \\ T(0) = \sin 2x \cos y \end{array} \right. \quad (3)$$

Решением следующих краевых задач

$$\begin{cases} \Delta X + \mu X = 0 & 0 < x < \pi \\ X|_{x=0} = X|_{x=\pi} = 0 \end{cases} \quad (4)$$

$$\begin{cases} \Delta Y + \mu Y = 0 & 0 < y < \pi \\ \partial_y Y|_{y=0} = \partial_y Y|_{y=\pi} = 0 \end{cases} \quad (5)$$

являются

$$\begin{cases} X_n = \sin nx & \mu = n^2 & n = 0, 1, \dots \\ Y_m = \cos my & \mu = m^2 & m = 0, 1, \dots \end{cases} \quad (6)$$

Для удобства выпишем решение для $V_{nm}(x, y)$:

$$\begin{cases} V_{nm}(x, y) = \sin nx \cos my, & n = 0, 1, \dots, m = 0, 1, \dots \\ \lambda_{nm} = \mu + \nu = n^2 + m^2 \end{cases} \quad (7)$$

Запишем постановку задачи для $T_{nm}(t)$:

$$\begin{cases} \partial_t T + \lambda_{nm} T = e^t \sin x \cos y \\ T(0) = \sin 2x \cos y \end{cases}, \quad (8)$$

где $\lambda_{nm} = \mu + \nu = n^2 + m^2$. Решим задачу Коши:

$$f_{nm} = \frac{1}{||V_{nm}||^2} \iint F(x, y, t) V_{nm}(x, y) dx dy = I \quad (9)$$

$$\phi_{nm} = \frac{1}{||V_{nm}||^2} \iint \Phi(x, y, t) V_{nm}(x, y) dx dy = J \quad (10)$$

$$I = \frac{4e^t}{\pi^2(1 + \delta_{n0})(1 + \delta_{m0})} \int_0^\pi \sin x \sin nx dx \int_0^\pi \cos y \cos my dy = e^t \quad (11)$$

$$f_{11} = e^t, \quad f_{nm} = 0, \quad n, m \neq 0 \quad (12)$$

$$J = \frac{4}{\pi^2(1 + \delta_{n0})(1 + \delta_{m0})} \int_0^\pi \sin 2x \sin nx dx \int_0^\pi \cos y \cos my dy = 1 \quad (13)$$

$$\phi_{21} = 1, \quad \phi_{nm} = 0, \quad n, m \neq 0 \quad (14)$$

В таком случае решение задачи Коши:

$$\begin{aligned} T_{nm} &= \phi_{nm} e^{-\lambda_{nm}t} + \int_0^t e^{-\lambda(t-\tau)} f_{nm}(\tau) d\tau = \phi_{21} e^{-\lambda_{21}t} + \int_0^t e^{-\lambda_{11}(t-\tau)} f_{11}(\tau) d\tau = \\ &= e^{-5t} + \int_0^t e^{-2(t-\tau)} e^\tau d\tau = e^{-5t} + \frac{e^t - e^{-2t}}{3} \end{aligned} \quad (15)$$

Полное аналитическое решение:

$$u(x, y, t) = \sum_m \sum_n T_{nm}(t) V_{nm}(x, y) = e^{-5t} \sin 2x \cos y + \frac{e^t - e^{-2t}}{3} \sin x \cos y \quad (16)$$

III. Построение разностной схемы

Введём одномерные разностные временную и пространственные сетки в области $\Omega = G \otimes [0, T]$:

$$\begin{aligned} G: \{(x, y) \mid x \in (0; \pi), y \in (0; \pi)\} \\ \omega_h: \{(x_i, y_j) \mid x_i = ih_x, i = \overline{0, N_x}, N_x h_x = \pi; y_j = jh_y, j = \overline{0, N_y}, N_y h_y = \pi\} \\ \omega_\tau: t_k = k\tau, k = \overline{0, N_\tau}, N_\tau \tau = T \\ \omega_{h\tau} = \omega_h \otimes \omega_\tau, \end{aligned} \quad (17)$$

где N_τ, N_x, N_y – число узлов сетки, h_x, h_y, h_τ – шаги по осям x, y, t соответственно. На введённой сетке будем рассматривать сеточные функции U_{ij}^k . Введём разностную аппроксимацию оператора Лапласа:

$$\Delta U_{ij}^k = \Lambda_x U_{ij}^k + \Lambda_y U_{ij}^k \quad (18)$$

$$\begin{cases} \Lambda_x U_{ij}^k = \frac{U_{(i+1)j}^k - 2U_{ij}^k + U_{(i-1)j}^k}{h_x^2} \\ \Lambda_y U_{ij}^k = \frac{U_{i(j+1)}^k - 2U_{ij}^k + U_{i(j-1)}^k}{h_y^2} \end{cases} \quad (19)$$

Теперь аппроксимируем уравнение теплопроводности из задачи следующим разностным уравнением:

$$\frac{U_{ij}^{k+1} - U_{ij}^k}{\tau} = \Lambda(\sigma U_{ij}^{k+1} + (1 - \sigma)U_{ij}^k) + f_{ij}^{k+\frac{1}{2}}, \quad (20)$$

где $f_{ij}^{k+\frac{1}{2}} = e^{t^{k+\frac{1}{2}}} \sin x \cos y$.

Начальное условие для функции U_{ij}^k получаем непосредственно из $u|_{t=0} = \sin 2x \cos y$:

$$U_{ij}^0 = \sin 2ih_x \cos jh_y \text{ для всех } i = \overline{0, N_x}, j = \overline{0, N_y} \quad (21)$$

Граничные условия Дирихле аппроксимируются точно:

$$U_{0j}^k = U_{N_x j}^k = 0 \text{ для всех } j = \overline{0, N_y} \quad (22)$$

Для граничных условий Неймана введём аппроксимацию односторонней разностной производной:

$$\begin{cases} \frac{U_{i1}^k - U_{i0}^k}{h_y} = 0 \\ \frac{U_{iN_y}^k - U_{i(N_y-1)}^k}{h_y} = 0 \end{cases} \text{ для всех } i = \overline{0, N_x} \quad (21)$$

Для граничных условий Дирихле укажем значения:

$$U_{1j}^k = U_{N_x j}^k = 0, \quad \begin{cases} k = \overline{0, N_\tau} \\ j = \overline{0, N_y} \end{cases} \quad (22)$$

Неявная схема имеет тот же порядок точности, что и явная, однако она безусловно устойчива, поэтому будем пользоваться ей. Разностная аппроксимация уравнения (1) в схеме переменных направлений имеет вид:

$$\frac{U_{ij}^{k+\frac{1}{2}} - U_{ij}^k}{0.5\tau} = \Lambda_x U_{ij}^{k+\frac{1}{2}} + \Lambda_y U_{ij}^k + f_{ij}^{k+\frac{1}{2}}, \quad (23)$$

$$\frac{U_{ij}^{k+1} - U_{ij}^{k+\frac{1}{2}}}{0.5\tau} = \Lambda_x U_{ij}^{k+\frac{1}{2}} + \Lambda_y U_{ij}^{k+1} + f_{ij}^{k+\frac{1}{2}}, \quad (24)$$

Переход от слоя j к слою $j + 1$ совершается в два этапа с шагами 0.5τ : сначала решается уравнение (23), неявное по направлению x и явное по направлению y , а затем уравнение (24), явное по направлению x и неявное по направлению y . Значение $U_{ij}^{k+\frac{1}{2}}$ является промежуточным и играет вспомогательную роль. Схема переменных направлений безусловно устойчива при любых шагах h_x, h_y и τ .

Используя явный вид разностных операторов, аппроксимирующих оператор Лапласа, приходим к краевой задаче:

$$0.5\tau\gamma_x U_{i-1,j}^{k+\frac{1}{2}} - (1 + \gamma_x)U_{i,j}^{k+\frac{1}{2}} + 0.5\tau\gamma_x U_{i+1,j}^{k+\frac{1}{2}} = -F_{i,j}^{k+\frac{1}{2}} \quad (25)$$

$$U_{0,j}^{k+\frac{1}{2}} = 0 \quad U_{N_x,j}^{k+\frac{1}{2}} = 0, \quad (26)$$

где $\gamma_\alpha = \frac{\tau}{h_\alpha^2}$, $\alpha = x, y$ и

$$F_{i,j}^{k+\frac{1}{2}} = 0.5\gamma_y(U_{i,j-1}^k + U_{i,j+1}^k) + (1 - \gamma_y)U_{i,j}^k + 0.5\tau f_{i,j}^{k+\frac{1}{2}} \quad (27)$$

Чтобы перейти со слоя $k + \frac{1}{2}$ на слой k , необходимо решить краевую задачу:

$$0.5\tau\gamma_x U_{i,j-1}^{k+1} - (1 + \gamma_y)U_{i,j}^{k+1} + 0.5\gamma_y U_{i,j+1}^{k+1} = -F_{i,j}^{k+1} \quad (25)$$

$$U_{i,1}^{k+1} - U_{i,0}^{k+1} = 0 \quad U_{i,N_y}^{k+1} - U_{i,N_y-1}^{k+1} = 0, \quad (26)$$

где

$$F_{i,j}^{k+1} = 0.5\gamma_x \left(U_{i-1,j}^{k+\frac{1}{2}} + U_{i+1,j}^{k+\frac{1}{2}} \right) + (1 - \gamma_x)U_{i,j}^{k+\frac{1}{2}} + 0.5\tau f_{i,j}^{k+\frac{1}{2}} \quad (27)$$

Как и в предыдущем случае, она решается методом прогонки при каждом фиксированном $i = \overline{0, N_x - 1}$. В результате получаем значение $F_{i,j}^{k+1}$ на новом слое. При переходе от слоя $j + 1$ к слою $j + 2$ процедура повторяется.

IV. Метод прогонки

Рассмотрим первый полуслой и будем решать задачу методом прогонки. Пусть имеется уравнение с начально-граничными условиями:

$$\begin{cases} A_i y_{i-1} - B_i y_i + C_i y_{i+1} = F_i, & |B_i| = |A_i| + |C_i| \\ y_0 = \alpha_1 y_1 + \beta_1, & i = \overline{0, N} \\ y_N = \alpha_2 y_{N-1} + \beta_2 \end{cases} \quad (28)$$

Пусть значение искомой функции в двух соседних точках связаны следующим линейным соотношением: $y_{i-1} = d_i y_i + \delta_i$. Тогда:

$$A_i(d_i y_i + \delta_i) - B_i y_i + C_i y_{i+1} = F_i \quad (29)$$

$$(A_i d_i - B_i) y_i = -C_i y_{i+1} + F_i - \delta_i A_i \quad (30)$$

$$y_{i+1} [(A_i d_i - B_i) d_{i+1} + C_i] = F_i - \delta_i A_i - (A_i d_i - B_i) \delta_{i+1} \quad (31)$$

Для того, чтобы это соотношение было верно для любых y_{i+1} нужно, чтобы выражение в квадратных скобках и правая часть было равны нулю. Приравнявая их нулю, получаем рекуррентные формулы для определения прогоночных коэффициентов:

$$d_{i+1} = \frac{C_i}{B_i - A_i d_i}, \quad \delta_{i+1} = \frac{F_i - \delta_i A_i}{A_i d_i - B_i} \quad (32)$$

Сравнивая граничные условия задачи с выражением $y_{i-1} = d_i y_i + \delta_i$, находим $d_1 = \alpha_1$, $\delta_1 = \beta_1$. Используя эти значения d_1 и δ_1 , совершим прогонку в направлении возрастания индекса, последовательно определяя значения коэффициентов d_i и δ_i для $i = \overline{0, N}$. Также нетрудно получить рекуррентные соотношения для y_{N-1} и y_N : $y_{N-1} = d_N y_N + \delta_N$ и $y_N = \alpha_2 y_{N-1} + \beta_2$. Из этих соотношений получаем:

$$y_{N-1} = d_N y_N + \delta_N = d_N (\alpha_2 y_{N-1} + \beta_2) + \delta_N \Rightarrow y_N = \frac{\alpha_2 y_{N-1} + \beta_2}{1 - d_N \alpha_2} \quad (33)$$

При $d_1 = \alpha_1$ и условиях, наложенных при постановке задачи на $B_i, A_i, C_i, \alpha_{1,2}$ получаем, что знаменатель формуле выше положительный. Следовательно значение y_N определено. Используя найденное значение y_N , делаем обратную прогонку в сторону уменьшающихся значений индекса, последовательно определяя значения y_i .

V. Реализация метода переменных направлений

Код программы, реализующий численное решение поставленной задачи, представлен в Приложении 1. Программа написана на языке Python. Количество шагов по осям координат задано $N_x = N_y = 500$ (шаг $h_x = h_y \approx 0.0063$), по оси времени – $N_\tau = 500$, а эволюция системы рассчитывается до 3 секунд (шаг $\tau = 0.006$).

VI. Результаты

Разобьём весь временной промежуток на две части: первая секунда и оставшееся время. На первом происходит затухание начального состояния системы, на втором видно только действие источника тепла.

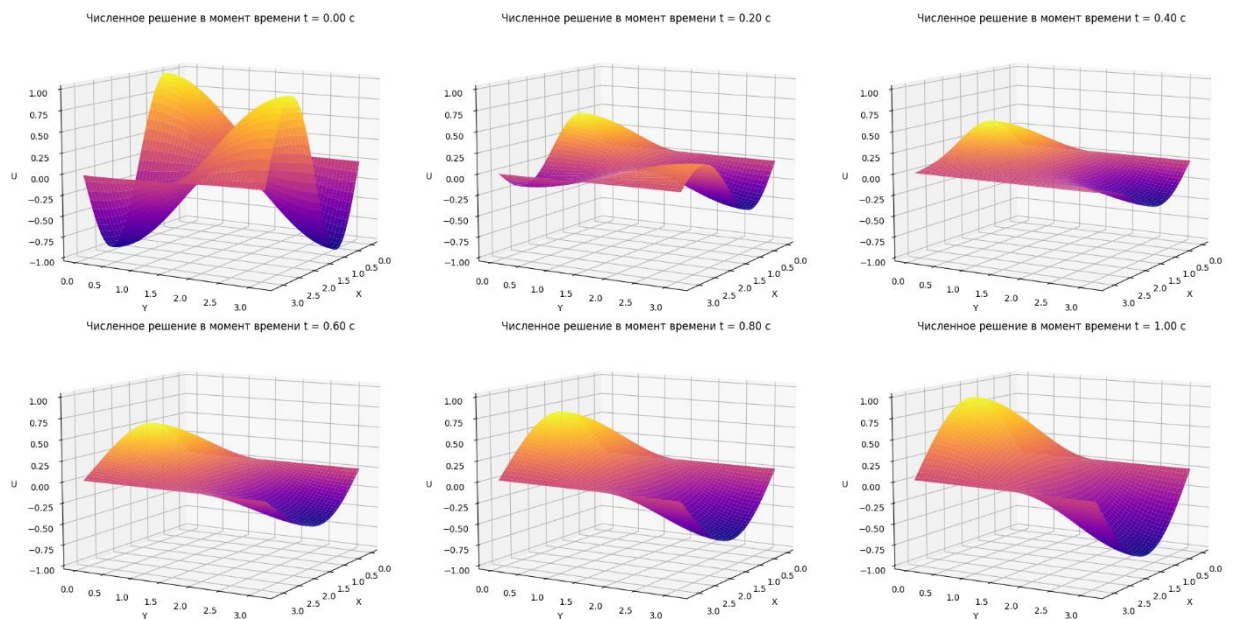


Рис. 1. Визуализация численного решения для первого временного промежутка в трёхмерном виде

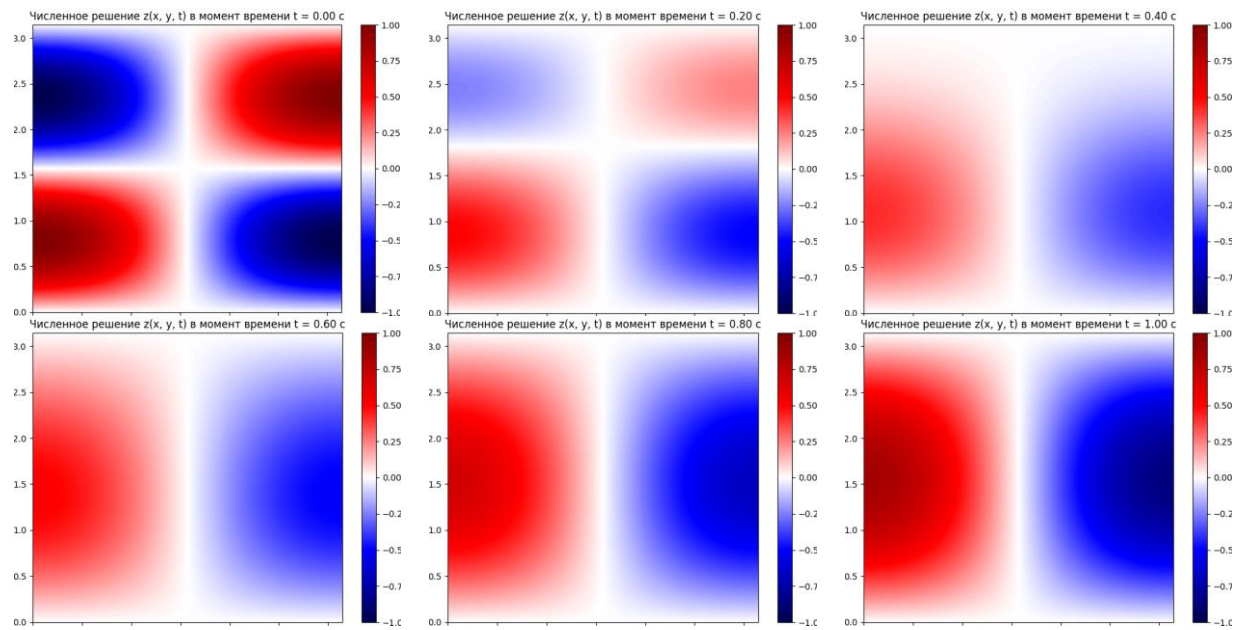


Рис. 2. Визуализация численного решения для первого временного промежутка в двумерном виде

Рассмотрим второй временной промежуток.

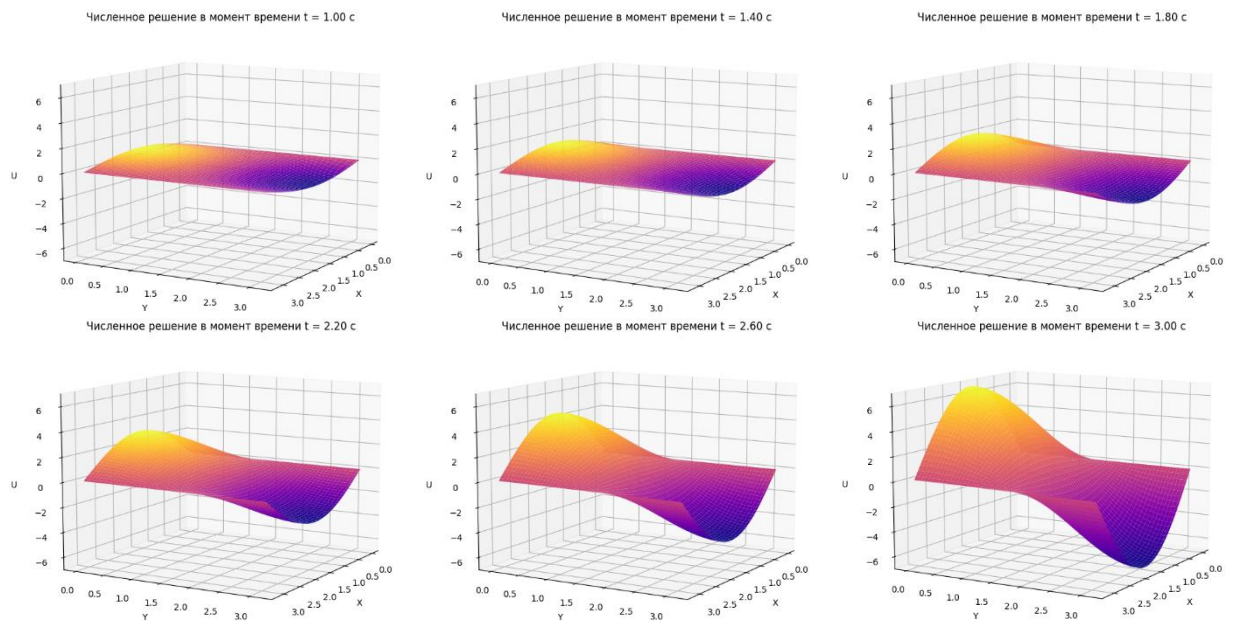


Рис. 3. Визуализация численного решения для второго временного промежутка в трёхмерном виде

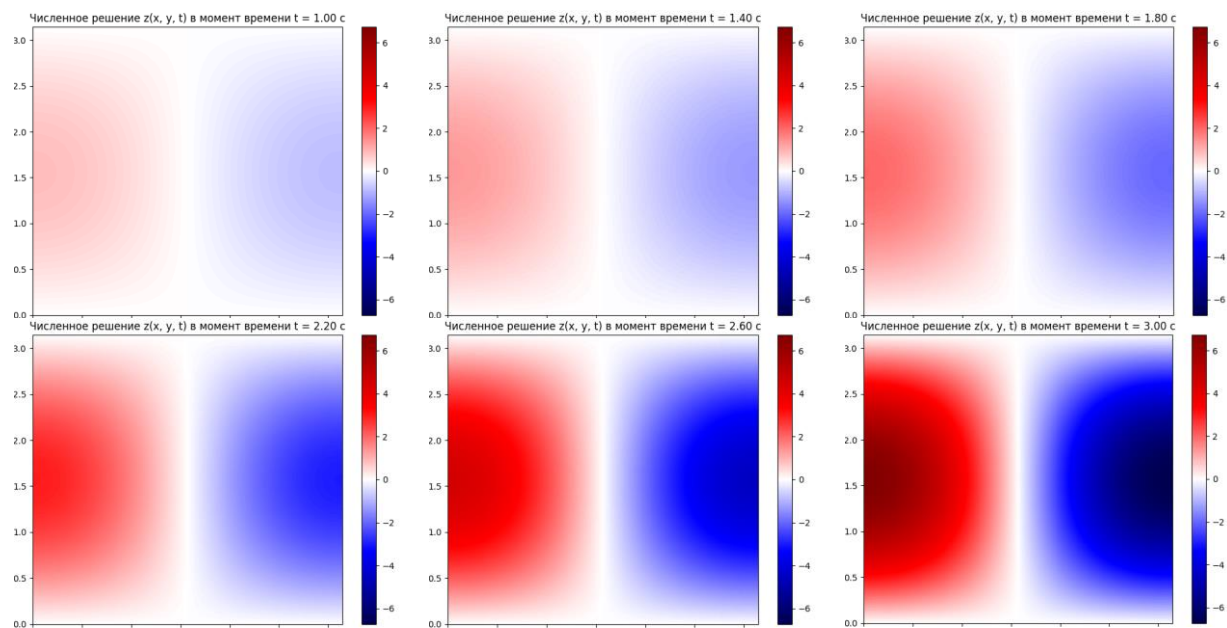
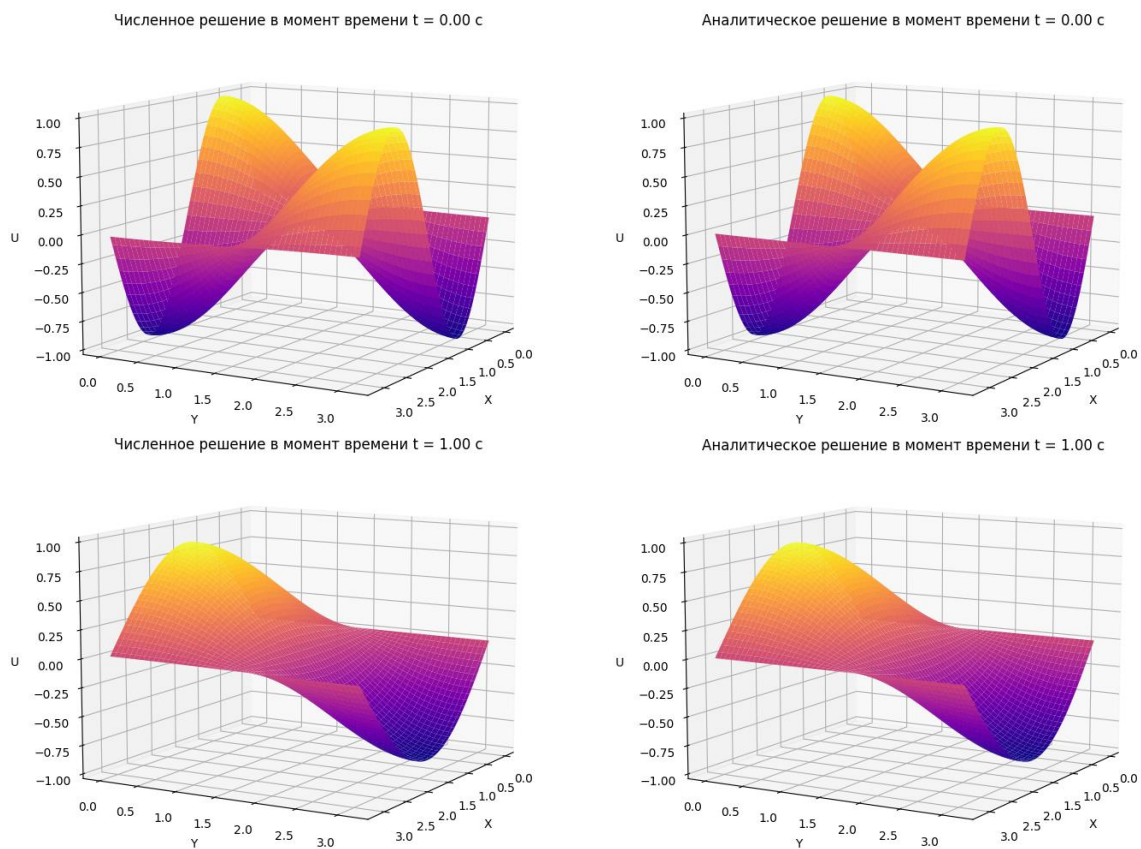


Рис. 4. Визуализация численного решения для второго временного промежутка в двумерном виде

VII. Сравнение результатов с аналитическим решением

Для верификации сравним визуализации для численного моделирования с аналитическим решением, полученным в части II этой работы.



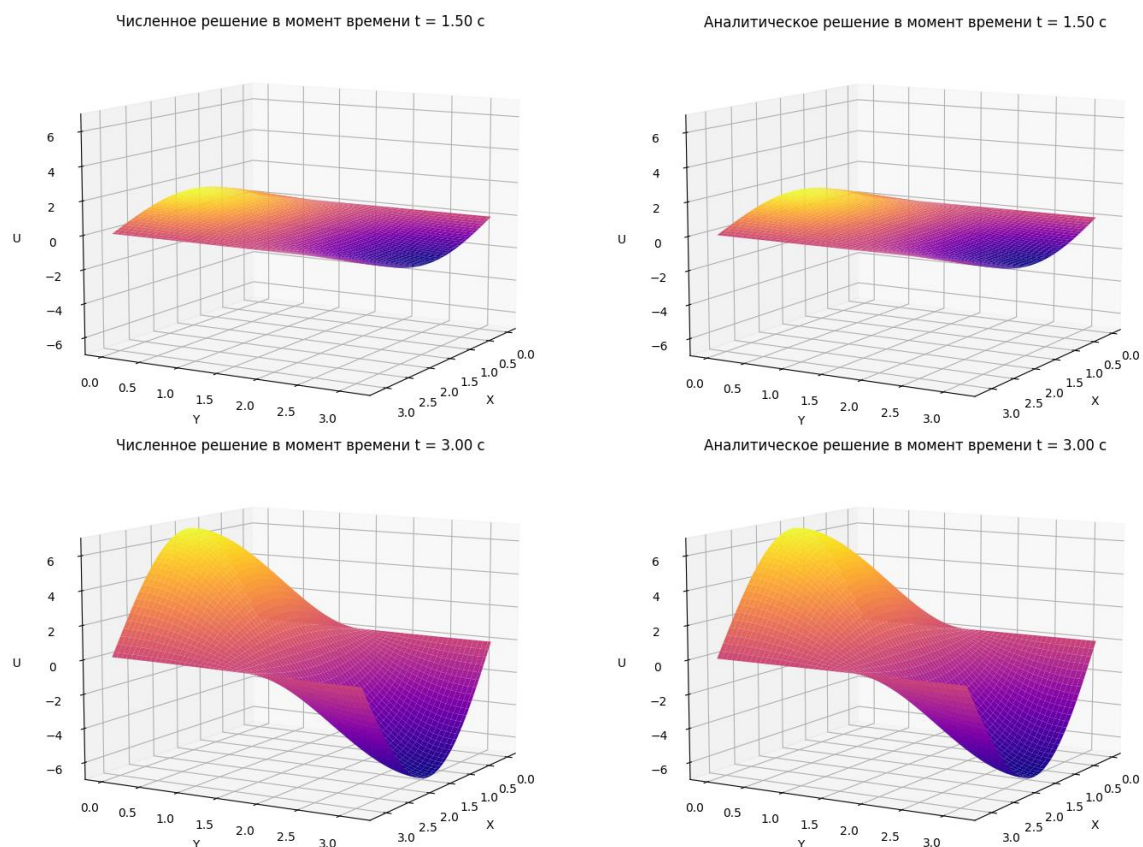


Рис. 5. Визуальное сравнение графиков аналитического и численного решений

Такое сравнение ненаглядно, поэтому посчитаем относительную ошибку для тех же моментов времени, которые были использованы в рисунках 1-4.

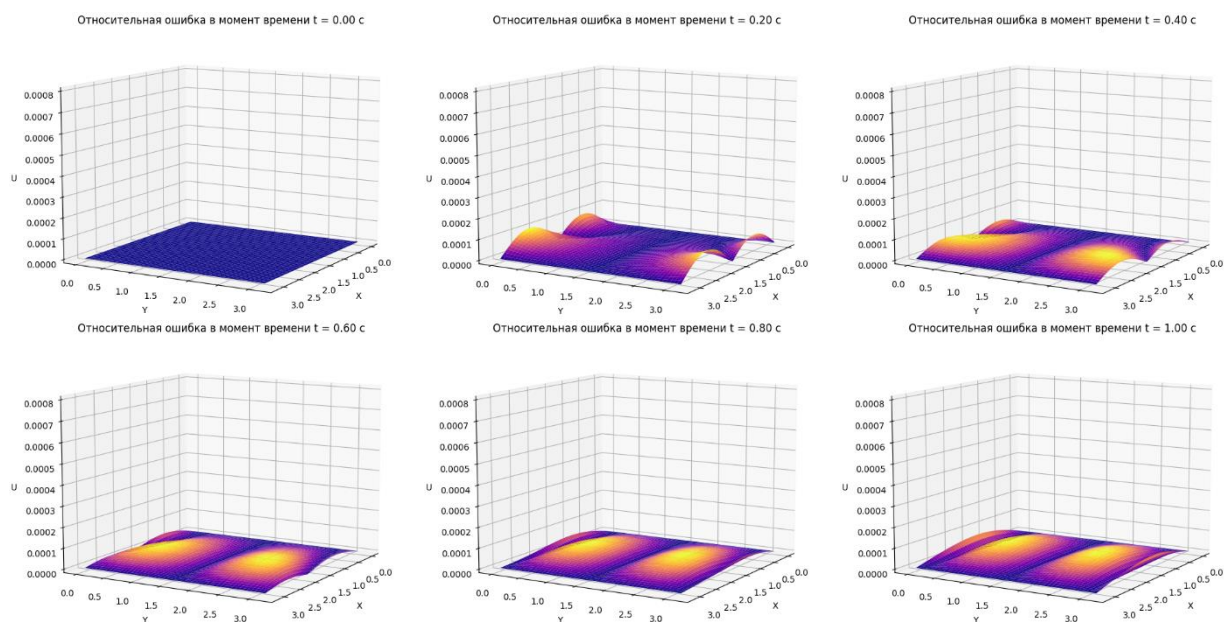


Рис. 6. График относительной ошибки для первого промежутка времени в трёхмерном виде

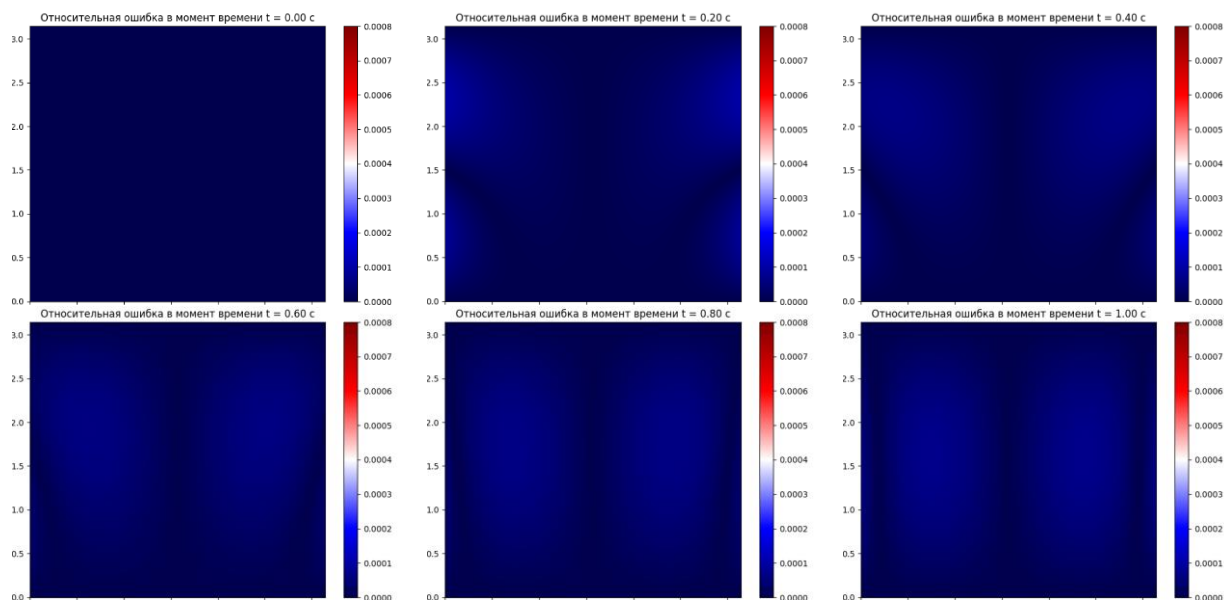


Рис. 7. График относительной ошибки для первого промежутка времени в двумерном виде

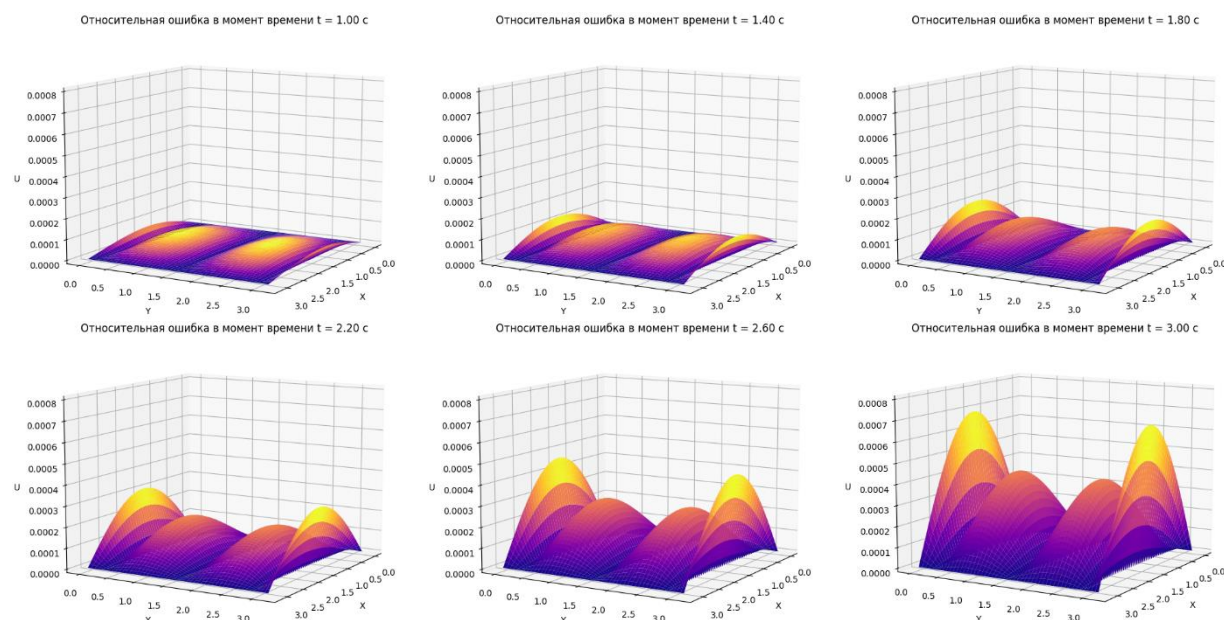


Рис. 8. График относительной ошибки для второго промежутка времени в трёхмерном виде

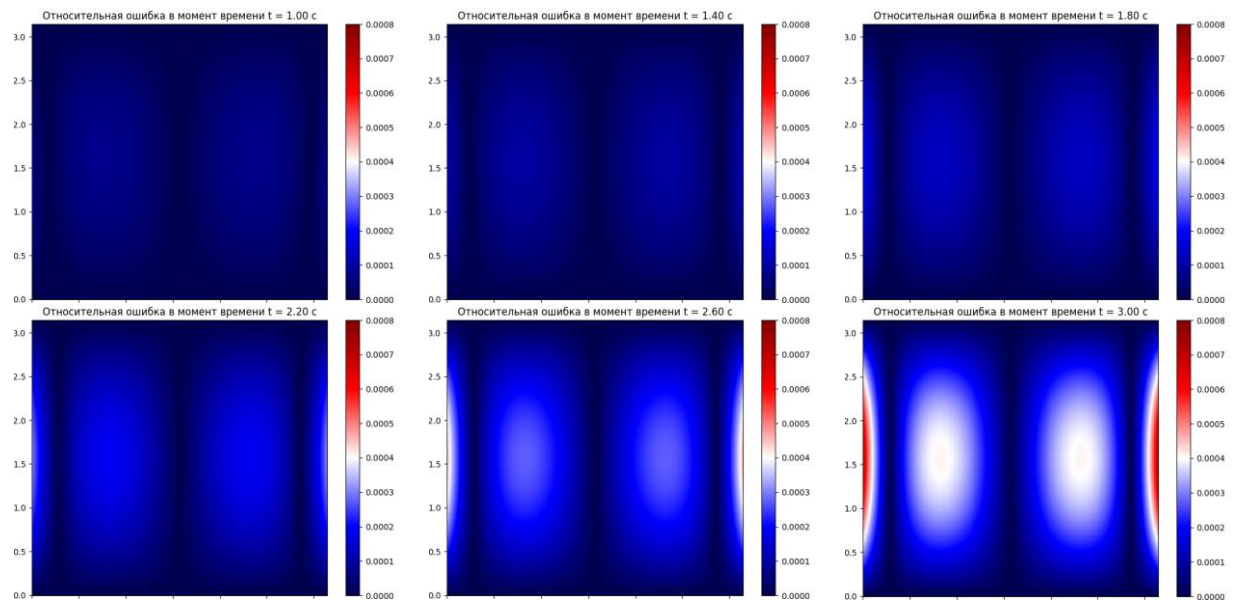


Рис. 9. График относительной ошибки для второго промежутка времени в двумерном виде

На графиках наглядно видно, что с ростом времени растёт и ошибка, однако максимальная относительная ошибка на рассматриваемом временном промежутке составила 0.01%, поэтому можно считать численное решение верным.

VIII. Заключение

В настоящей работе было проведено численное решение краевой с использованием метода переменных направлений. Постановка задачи приведена в пункте I. В пунктах III и IV была построена разностная схема и описан метод прогонки. В пункте V приведён код программы, реализующий этот способ решения. Для верификации результатов использовалось сравнение с аналитическим решением, полученным в пункте II. По итогам сравнения было установлено, что на временном промежутке до 3 секунд относительная ошибка составила менее 0.01%. Её можно снижать и дальше, увеличивая количество шагов сетки, однако даже используемая сетка $500 \times 500 \times 500$ занимает много времени для визуализации результатов.

```

import time
import numpy as np
import matplotlib.pyplot as plt
from numpy import exp, sin, cos, pi, max
from numba import njit
from tqdm import tqdm
import imageio
from PIL import Image

Nx = 500
Ny = 500
M = 500
T = 3
x = np.linspace(0, pi, Nx)
y = np.linspace(0, pi, Ny)
t = np.linspace(0, T, M)
h_x = x[1] - x[0]
h_y = y[1] - y[0]
tau = t[1] - t[0]
gamma_x = tau / (h_x ** 2)
gamma_y = tau / (h_y ** 2)
u = np.zeros((Nx, Ny, 2 * M + 1))

saving = True

@njit
def F_1(i1, i2, j):
    return 0.5 * gamma_y * (u[i1, i2 - 1, j - 1] + u[i1, i2 + 1, j - 1]) + (1 - gamma_y) * u[
        i1, i2, j - 1] + 0.5 * tau * exp(tau * (j + 1) / 2) * sin(x[i1]) * cos(y[i2])

@njit
def F_2(i1, i2, j):
    return 0.5 * gamma_x * (u[i1 - 1, i2, j - 1] + u[i1 + 1, i2, j - 1]) + (1 - gamma_x) * u[
        i1, i2, j - 1] + 0.5 * tau * exp(tau * (j - 1) / 2) * sin(x[i1]) * cos(y[i2])

@njit
def progonka_x(i2, j, u_new):
    d = np.zeros(Nx)
    sigma = np.zeros(Nx)
    d[1] = 0
    sigma[1] = 0
    A = 0.5 * gamma_x
    B = 1 + gamma_x
    C = 0.5 * gamma_x
    for m in range(1, Nx - 1):
        Fm = - F_1(m, i2, j)
        d[m + 1] = C / (B - A * d[m])
        sigma[m + 1] = (Fm - A * sigma[m]) / (A * d[m] - B)
    u_new[Nx - 1, i2, j] = 0
    for m in range(Nx - 1, 0, -1):
        u_new[m - 1, i2, j] = d[m] * u_new[m, i2, j] + sigma[m]
    return u_new

@njit
def progonka_y(i1, j, u_new):
    d = np.zeros(Ny)
    sigma = np.zeros(Ny)
    d[1] = 1
    sigma[1] = 0
    A = 0.5 * gamma_y
    B = 1 + gamma_y
    C = 0.5 * gamma_y
    for m in range(1, Ny - 1):
        Fm = - F_2(i1, m, j)
        d[m + 1] = C / (B - A * d[m])
        sigma[m + 1] = (Fm - A * sigma[m]) / (A * d[m] - B)
    u_new[i1, Ny - 1, j] = sigma[-1] / (1 - d[-1])
    for m in range(Ny - 1, 0, -1):
        u_new[i1, m - 1, j] = d[m] * u_new[i1, m, j] + sigma[m]
    return u_new

# начальное условие
for i in range(0, Nx):
    for j in range(0, Ny):
        u[i, j, 0] = sin(2 * x[i]) * cos(y[j])

for j in tqdm(range(1, 2 * M, 2)):

```

```

        for i2 in range(1, Ny - 1):
            u = progonka_x(i2, j, u)
        for i1 in range(1, Nx - 1):
            u = progonka_y(i1, j + 1, u)

max_value = np.max(abs(u))

x_grid, t_grid = np.meshgrid(x[1:-2], y[1:-2])
plt.ion()
fig = plt.figure(figsize=(8, 6))
for timer in tqdm(range(M), desc='График численного решения'):
    plt.clf()
    ax_3d = plt.subplot(projection='3d')
    ax_3d.plot_surface(x_grid, t_grid, u[1:-2, 1:-2, timer*2].T, cmap='plasma')
    ax_3d.set_xlabel('X')
    ax_3d.set_ylabel('Y')
    ax_3d.set_zlabel('U')
    if t[timer] < 1:
        ax_3d.set_zlim(-1, 1)
    else:
        ax_3d.set_zlim(-max_value, max_value)
    ax_3d.view_init(elev=10, azim=30)
    ax_3d.dist = 8
    plt.title(f'Численное решение в момент времени t = {t[timer]:.2f} с')
    plt.draw()
    plt.gcf().canvas.flush_events()
    if saving:
        plt.savefig(f'2 Численное решение/{timer}.png')

z = np.zeros((Nx, Ny, M * 2 + 1))
for i in range(Nx):
    for j in range(Ny):
        for k in range(M):
            z[i, j, k] = exp(-5 * k*tau) * sin(2 * x[i]) * cos(y[j]) + (exp(k*tau) - exp(-2 *
k*tau)) / 3 * sin(x[i]) * cos(y[j])

x_grid, t_grid = np.meshgrid(x, y)
plt.ion()
fig = plt.figure(figsize=(8, 6))
for timer in tqdm(range(M), desc='График аналитического решения'):
    plt.clf()
    ax_3d = plt.subplot(projection='3d')
    ax_3d.plot_surface(x_grid, t_grid, z[:, :, timer].T, cmap='plasma')
    ax_3d.set_xlabel('X')
    ax_3d.set_ylabel('Y')
    ax_3d.set_zlabel('U')
    if t[timer] < 1:
        ax_3d.set_zlim(-1, 1)
    else:
        ax_3d.set_zlim(-max_value, max_value)
    ax_3d.view_init(elev=10, azim=30)
    ax_3d.dist = 8
    plt.title(f'Аналитическое решение в момент времени t = {t[timer]:.2f} с')
    plt.draw()
    plt.gcf().canvas.flush_events()
    if saving:
        plt.savefig(f'2 Аналитическое решение/{timer}.png')
plt.ioff()

x_grid, t_grid = np.meshgrid(x, y)
plt.ion()
fig = plt.figure(figsize=(8, 6))
for timer in tqdm(range(M), desc='График относительной ошибки'):
    plt.clf()
    ax_3d = plt.subplot(projection='3d')
    ax_3d.plot_surface(x_grid, t_grid, abs(u[:, :, timer*2].T - z[:, :, timer].T) / max(u),
cmap='plasma')
    ax_3d.set_xlabel('X')
    ax_3d.set_ylabel('Y')
    ax_3d.set_zlabel('U')
    ax_3d.set_zlim(0, 0.0008)
    ax_3d.view_init(elev=10, azim=30)
    ax_3d.dist = 8
    plt.title(f'Относительная ошибка в момент времени t = {t[timer]:.2f} с')
    plt.draw()
    plt.gcf().canvas.flush_events()
    if saving:
        plt.savefig(f'2 Относительная ошибка/{timer}.png')
plt.ioff()

```

```

plt.ion()
fig = plt.figure(figsize=(8, 6))
for timer in tqdm(range(M), desc='График численного решения'):
    plt.clf()
    if t[timer] < 1:
        plt.pcolormesh(y, x, u[:, :, timer*2], cmap='seismic', vmin=-1, vmax=1)
    else:
        plt.pcolormesh(y, x, u[:, :, timer*2], cmap='seismic', vmin=-max_value, vmax=max_value)
    plt.colorbar()
    plt.ylabel('Координата по X', fontsize=12)
    plt.xlabel('Координата по Y', fontsize=12)
    plt.title(f'Численное решение z(x, y, t) в момент времени t = {t[timer]:.2f} с')
    plt.draw()
    plt.gcf().canvas.flush_events()
    if saving:
        plt.savefig(f'2 Численное решение/pcolormesh_{timer}.png')
plt.ioff()

plt.ion()
fig = plt.figure(figsize=(8, 6))
for timer in tqdm(range(M), desc='График аналитического решения'):
    plt.clf()
    if t[timer] < 1:
        plt.pcolormesh(y, x, z[:, :, timer], cmap='seismic', vmin=-1, vmax=1)
    else:
        plt.pcolormesh(y, x, z[:, :, timer], cmap='seismic', vmin=-max_value, vmax=max_value)
    plt.colorbar()
    plt.ylabel('Координата по X', fontsize=12)
    plt.xlabel('Координата по Y', fontsize=12)
    plt.title(f'Аналитическое решение z(x, y, t) в момент времени t = {t[timer]:.2f} с')
    plt.draw()
    plt.gcf().canvas.flush_events()
    if saving:
        plt.savefig(f'2 Аналитическое решение/pcolormesh_{timer}.png')
plt.ioff()

plt.ion()
fig = plt.figure(figsize=(8, 6))
for timer in tqdm(range(M), desc='График относительной ошибки'):
    plt.clf()
    plt.pcolormesh(y, x, abs(u[:, :, timer*2]-z[:, :, timer]) / max(u), cmap='seismic', vmin=0,
vmax=0.0008)
    plt.colorbar()
    plt.ylabel('Координата по X', fontsize=12)
    plt.xlabel('Координата по Y', fontsize=12)
    plt.title(f'Относительная ошибка в момент времени t = {t[timer]:.2f} с')
    plt.draw()
    plt.gcf().canvas.flush_events()
    if saving:
        plt.savefig(f'2 Относительная ошибка/pcolormesh_{timer}.png')
plt.ioff()

# ----- СОЗДАЁМ АНИМАЦИИ -----
if saving:
    frames = []
    for t in range(M):
        image = imageio.v2.imread(f'2 Численное решение/{t}.png')
        frames.append(image)
    imageio.mimsave('2 Численное решение.gif', # output gif
                    frames, # array of input frames
                    fps=30)

    frames = []
    for t in range(M):
        image = imageio.v2.imread(f'2 Аналитическое решение/{t}.png')
        frames.append(image)
    imageio.mimsave('2 Аналитическое решение.gif', # output gif
                    frames, # array of input frames
                    fps=30)

    frames = []
    for t in range(M):
        image = imageio.v2.imread(f'2 Относительная ошибка/{t}.png')
        frames.append(image)
    imageio.mimsave('2 Относительная ошибка.gif', # output gif
                    frames, # array of input frames
                    fps=30)

```

```

frames = []
for t in range(M):
    image = imageio.v2.imread(f'2 Численное решение/pcolormesh_{t}.png')
    frames.append(image)
imageio.mimsave('2 pcolormesh Численное решение.gif', # output gif
                frames, # array of input frames
                fps=30)

frames = []
for t in range(M):
    image = imageio.v2.imread(f'2 Аналитическое решение/pcolormesh_{t}.png')
    frames.append(image)
imageio.mimsave('2 pcolormesh Аналитическое решение.gif', # output gif
                frames, # array of input frames
                fps=30)

frames = []
for t in range(M):
    image = imageio.v2.imread(f'2 Относительная ошибка/pcolormesh_{t}.png')
    frames.append(image)
imageio.mimsave('2 pcolormesh Относительная ошибка.gif', # output gif
                frames, # array of input frames
                fps=30)

folders = ['2 Численное решение', '2 Аналитическое решение', '2 Относительная ошибка']
for folder in folders:
    for t in range(M):
        im = Image.open(f'{folder}/{t}.png')
        im = im.crop((70, 45, 690, 540))
        im.save(f'{folder}/{t}.png')

        im = Image.open(f'{folder}/pcolormesh_{t}.png')
        im = im.crop((70, 45, 690, 540))
        im.save(f'{folder}/pcolormesh_{t}.png')

```